

How We'll Work Together

Building agorasim.pt — our process, our roles, and what to expect. In plain English.

PREPARED FOR	Diogo & Rita — Agorasim (Sintra · Mafra · Ericeira)
PREPARED BY	Jamie Nisbet · jamie.nisbet@outlook.be
WORKING LINK	agorasim.jamienisbet.com (private build address — not public until launch)

1. Why this document exists

We are a small, focused team, and that is our advantage. I build the software; you two own the vision, test the result, and give it the final yes. This document explains how we turn your ideas into a finished website quickly and without confusion — so we all know who does what, and what happens next at every stage. No technical background is needed to read it: anything that sounds like jargon is explained in plain English in the glossary at the end.

2. The big picture — how a website actually gets built

Software isn't built all at once and revealed at the end like a painting. It's built in a repeatable loop that every single feature travels through. The industry calls this the **SDLC — the Software Development Life Cycle**. Stripped of the jargon, it's just five plain stages:

1. **Plan.** We agree what a feature should do and what “finished” looks like for it.
2. **Build.** I write the code and put the feature together.
3. **Check (my side).** I test it myself first. You should never be the first person to hit an obvious problem.
4. **Review (your side).** You try it on the working link and tell me what works and what doesn't.
5. **Sign-off & ship.** Once you're happy, that feature is locked in and we move to the next one.

If something isn't right at the Review stage, we simply loop back to Build and go round again. That loop is the whole engine. Everything below is just detail on how we run it well.

3. How we've chosen to work: one feature at a time

We are building the site **feature by feature**, in sequence — not all six at once in a big mysterious lump. Each feature is a **milestone** with a clear finish line. You review it, you sign it off, and only then do we start the next. This keeps feedback focused, means nothing is ever left half-built, and means you always know exactly what you're looking at.

Here is the roadmap — the six features from the offer, in a sensible build order. Booking and payments come early because they're the engine everything else supports; the order can flex if you'd prefer:

#	Milestone	What you'll be testing
1	Website + GEO foundation	The rebuilt bilingual (PT/EN) site — pages, look, speed, and being findable on Google and AI search.
2	Instant tour booking (Stripe)	Making a real test booking and payment on your own site, start to finish.

#	Milestone	What you'll be testing
3	Custom payment links	Deposit + balance links for car hire and events.
4	Email & SMS notifications	Confirmations, reminders and follow-ups arriving correctly to the guest.
5	AI automated blog	Articles about the Saloia region publishing themselves, on-brand.
6	Social generator + auto-poster	On-brand Instagram/Facebook posts generated and scheduled.

Note on going live: the public site at agorasim.pt does *not* change while we build. Everything happens on a private working address — agorasim.jamienisbet.com — which is where you'll test each feature. We only “flip the switch” to the real domain once the whole build is finished and you're happy with all of it.

4. The cycle for a single feature, step by step

Every milestone follows the same rhythm, so it becomes second nature after the first one:

6. **Kick-off.** I send a short note: what this feature does and what “done” means for it.
7. **I build and test it** on my own side until it's solid.
8. **Hand-over.** I tell you “Feature X is ready to test at agorasim.jamienisbet.com” and give you a short checklist.
9. **You test it** against that checklist, on both your phone and a computer, within a few days so we keep momentum.
10. **You send feedback** in one batch — everything you noticed, in a single list.
11. **I fix and adjust**, then hand it back for a quick re-check.
12. **You sign off.** The milestone is complete, and we start the next one.

5. Your responsibilities — Diogo & Rita

You are the product owners and the testers. The build moves as fast as you feed and check it. Concretely:

- **Be the source of truth.** Facts, prices, tour details, add-ons, photos, brand voice, contact details — these come from you.
- **Supply content promptly.** Text, images, logos and your list of tours/add-ons/prices. This is the single biggest thing that keeps us fast — a feature can't be finished around missing content.
- **Test when asked, within the window.** When a feature is ready, try it out within a few days so the project doesn't stall waiting on us.
- **Give clear, specific feedback** (see section 7).
- **Speak with one voice.** If the two of you disagree on something, settle it between you first, then send me one agreed decision — it saves a lot of back-and-forth.
- **Sign off each milestone** so we can move on with confidence.
- **Own your accounts.** Stripe, the domain, social logins — these stay in your names. I'll guide you through every setup step.

6. My responsibilities — Jamie

- **Build all six features** to the scope we agreed.
- **Test my own work first.** You should never be the one to find an obvious bug.

- **Keep you informed.** What's ready, what's next, and anything that's blocking me.
- **Make testing easy** with a short, plain checklist for each feature.
- **Fix genuine bugs at no cost.** If something in the agreed scope doesn't work as it should, fixing it is on me.
- **Keep it secure.** Your site, data and accounts set up and protected properly.
- **Be straight about scope and timing.** I'll always tell you honestly whether something is part of the build or a new request.

7. How to test — and how to give feedback that speeds us up

Testing a feature

- Open the link on both your phone and a computer — most guests will book on a phone.
- **Complete the real task end to end** (e.g. actually make a test booking, don't just look at the page).
- Try the “wrong” things too: leave a field blank, pick an odd date, hit back. See if it copes gracefully.
- Check both the Portuguese and the English.
- Note anything confusing, not only anything broken.

Giving feedback

The most useful feedback tells me three things: **what you did, what you expected, and what actually happened** (a screenshot is gold).

Vague: “The booking is broken.”

Useful: “On my iPhone, I chose 4 guests and 5 August, tapped Pay, and it spun forever and never confirmed. Screenshot attached.”

And please **batch it** — one list per feature beats twenty separate messages through the day. It lets me fix everything in one focused pass.

8. New ideas, changes, and “wouldn't it be cool if...”

New ideas are welcome and often good — but timing matters. The six features are the agreed build. When a fresh idea appears mid-milestone, we **park it on a “later” list** rather than let it derail the feature in front of us. This protects the timeline and keeps things fair for both sides.

It also helps to keep two things separate: a **bug** (something in scope that doesn't work — I fix it at no cost) versus a **new request** (something we didn't scope — we note it, and larger additions can become a future piece of work). I'll always be transparent about which bucket something falls into.

9. Our rhythm and timeline

Because we work milestone by milestone, you'll feel steady, visible progress rather than a long silence followed by a big reveal. Each milestone runs through the cycle in section 4, with a short review window for you. I'll keep a simple running status so you always know which feature is in build, which is waiting on your review, and which is signed off.

The one rule that decides our speed: content supplied quickly + features tested promptly + clear, batched feedback = a fast launch. That part is genuinely in your hands.

10. Tech jargon — a plain-English glossary

You'll hear some of these from me. None of them are complicated once translated:

Term	In plain English
SDLC	Software Development Life Cycle — the repeatable Plan → Build → Check → Review → Ship loop every feature travels through.
Milestone	One finished, signed-off feature. Our project is six of them in a row.
Scope	The agreed list of what's being built. “Out of scope” means it's a new request, not part of the deal.
Bug	Something in scope that doesn't work as intended. I fix these at no cost.
Feature	A distinct piece of functionality — e.g. “instant booking” or “the blog.”
Frontend	The part you and guests see and click — pages, buttons, forms, design.
Backend	The engine behind the scenes — where bookings, payments and data are handled.
Deploy / Deployment	Publishing a new version of the site so it's live at a web address.
Staging vs Production	Staging = a private practice version for testing. Production = the real, public site. During our build, agorasim.jamienisbet.com is your test version.
Preview link	A temporary private web address showing a specific version of the site for review.
Domain / DNS	Your web address (agorasim.pt) and the system that points it at your site.
SSL / HTTPS	The padlock in the browser — encryption that keeps the site and payments secure.
Responsive / mobile-first	The site reshaping itself to look right on any screen, phone first.
Static site (SSG)	The site is pre-built into fast, fixed pages — quick to load and cheap to run.
SEO / GEO	Being found by search. SEO = classic Google ranking; GEO = being cited by AI search tools like ChatGPT.
JSON-LD / hreflang	Invisible labels in the page that tell Google and AI what your business is, and which pages are Portuguese vs English.
CMS / content	The words, images and information on the site — the part you own and supply.
Stripe	The service that securely takes card payments on your site.
Stripe Connect	The Stripe setup that lets the platform's commission be split off automatically at payment time.
Application fee	The technical name for that automatic commission split inside Stripe.
Payout	Stripe moving money from your Stripe balance to your actual bank account.
Webhook	An automatic “ping” between systems — e.g. Stripe telling the site “this booking is paid” so a confirmation can be sent.
API	A doorway that lets two software systems talk to each other automatically.
Repository / repo	The organised store of all the site's code and its full history of changes.
UAT	User Acceptance Testing — the fancy name for the testing you two do before sign-off.
Sign-off	Your “yes, this is good” that completes a milestone.

11. In short

I build in a steady loop, one feature at a time. You supply the content, test each feature on the working link, and give clear feedback so we can lock it in and move on. Keep that loop turning quickly and agorasim.pt goes from brochure to booking engine faster than you'd expect — with no surprises along the way.

Jamie Nisbet · jamie.nisbet@outlook.be